

Actividad Consulta en Casa



**INSTITUCIÓN UNIVERSITARIA
COLEGIO MAYOR DEL CAUCA**

Presentado por:

Cristóbal Alexander Chantre Vernaza

Presentado a:

Víctor Hugo Pinto Rodríguez

Materia: Electiva I

Semestre: VI

Institución Universitaria Colegio Mayor del Cauca

Facultad de Ingeniería

Ingeniería Informática

Popayán-Cauca

2026

1. ¿Qué son los hooks en React?

Los hooks son funciones especiales (que empiezan siempre por **use**) que permiten realizar tareas concretas dentro del ecosistema de React. Existen muchos hooks, muchos de ellos integrados en React, pero también es posible crear nuestros propios hooks personalizados.

1.1 Los objetivos que persiguen los hooks son los siguientes:

- Simplificar la complejidad y conseguir componentes más sencillos.
- Reutilizar lógica javascript y su estado (información que cambia)
- Organizar mejor nuestro código y escribir menos líneas.

1.2 Reglas fundamentales:

Los hooks de React tienen algunas reglas principales que hay que cumplir:

- Los hooks se llaman en el nivel superior. Nunca en condicionales, bucles o funciones anidadas.
- Solo se pueden llamar componentes funcionales o hooks. Nunca en funciones normales de javascript.

2. El hook en useState

Probablemente, el hook más sencillo de mostrar como ejemplo sea **useState**.

El **hook useState** permite añadir y gestionar un estado en componentes funcionales, algo que sólo era posible antiguamente con las clases de Javascript. Si el concepto estado es nuevo para ti, cuando hablamos de estado nos referimos a la información que cambia en un componente.

El ejemplo más clásico de estado es la creación de un contador, que contiene un estado (que cambia) que es el número del contador. Veamos un ejemplo de código:

```
function Counter() {  
  const [counter, setCounter] = useState(0);  
  
  return (  
    <div>  
      <p>El contador está en: {counter}</p>  
      <button onClick={() => setCounter(counter + 1)}>Aumentar</button>  
    </div>  
  );  
}
```

Código 2.1 Ejemplo de código (para crear un contador)

Recordemos que los hooks son simplemente funciones. En la primera línea estamos ejecutando el hook **useState()** y nos devuelve un array con dos elementos, que estamos desestructurando para tenerlo en dos variables separadas.

- El primer elemento lo hemos llamado **counter** y es la información actual del contador, el estado.

- El segundo elemento lo hemos llamado **setCounter** y es una función para actualizar ese estado.

Por norma general, los hooks **useState** utilizan esta forma de nombrar, donde el segundo elemento suele comenzar por set, seguido del nombre del primer elemento, utilizando camelCase.

Además, observa que el hook `useState()` se usa pasando el valor 0 por parámetro. Este valor es el estado inicial de nuestro counter. Ahora que tenemos claro esto, podemos entender mejor el uso de `counter` y `setCounter` en la parte del JSX.

Aplicando el ejemplo del código anterior [\(Código 2.1\)](#)

```
Contador.tsx X
s13-inicio-react > 02-ejercicio-contador > src > components > Contador.tsx > ...
1  import { useState } from 'react';
2
3  export const Contador = () => {
4    // Declaramos el useState para controlar el número del contador
5    const [contador, setContador] = useState<number>(0);
6
7    // Funciones para modificar el valor
8    const sumar = () => setContador(contador + 1);
9    const restar = () => setContador(contador - 1);
10   const resetear = () => setContador(0);
11
12   return (
13     <div style={{ padding: '20px' }}>
14       <h3>Ejercicio 1: Contador</h3>
15
16       {/* Se renderiza el valor dinámico */}
17       <h1>Valor: {contador}</h1>
18
19       {/* Botones con sus eventos onClick */}
20       <button onClick={sumar}>Sumar (+1)</button>
21       <button onClick={restar} style={{ margin: '0 10px' }}>Restar (-1)</button>
22       <button onClick={resetear}>Resetear</button>
23     </div>
24   );
25 };
```

Código 2.2 Contador creado



Figura 2.1 Visualización del contador

3. El hook en useEffect

En React, los efectos secundarios se gestionan mediante el hook **useEffect**, pero antes de empezar a trabajar con ellos necesitamos entender sus bases. En programación, llamamos efecto secundario (o side-effect) a cualquier acción que realiza nuestro código (en el interior de una función, por ejemplo) y afecta a algo que se encuentra fuera del ámbito de esa función, es decir, que estamos alterando algo no relacionado directamente con lo que devolvemos en la función.

3.1 Efectos secundarios

Para entender mejor el concepto de efecto secundario de la programación, imaginemos esta sencilla función de un contador:

```
let counter = 0;

function incrementCounter() {
  counter++; // Modifica la variable global (efecto secundario)
}

incrementCounter();
console.log(counter); // 1
```

Código 3.1 Efecto Secundario (Función incrementar contador)

Dicha función está alterando el valor de counter, el cuál no pertenece a la función incrementCounter(), por lo que aunque está realizando nuestro objetivo correctamente (incrementar el contador), se está realizando mediante un efecto secundario.

Algunos ejemplos comunes de efectos secundarios en JavaScript podrían ser:

- Modificar variables fuera del ámbito de su función (ej: variables globales)
- Realizar una operación de entrada/salida (ej: escribir en un archivo, petición a una API...)
- Actualizar y modificar elementos del DOM (ej: cambios en la estructura de la página)
- Crear temporizadores (ej: setTimeout o setInterval)
- Suscribirse a eventos (ej: escuchar un click u otra acción)

3.2 Funciones Puras

El ejemplo anterior se puede reescribir intentando que tus funciones sean puras. Una función pura es una función que cumple estas condiciones:

- Dado un argumento por parámetro, siempre devuelve el mismo resultado (es determinista).
- No muta nada fuera de su ámbito (no tiene efectos secundarios).

Veamos el ejemplo anterior, cambiando la función incrementCounter() para que sea una función pura, y evitando la creación de efectos secundarios:

```
let counter = 0;

function incrementCounter(counter) {
  return counter + 1;
}

counter = incrementCounter(counter);
console.log(counter); // 1
```

Código 3.2 Ejemplo de cómo evitar efectos secundarios devolviendo un valor modificado en lugar de mutar variables externas.

Observa que en lugar de mutar la variable externa `counter` en la función, lo que hacemos es devolver una versión modificada de los datos que recibimos. Esto ayuda considerablemente a no crear efectos secundarios y a que nuestro código sea más predecible y evitemos bugs accidentales.

3.3 Side effects en React

En el ecosistema de React, el concepto efecto secundario (muchas veces abreviado como efecto) aparece mucho y también se suele relacionar con las acciones que tienen consecuencias fuera de la función o componente en la que se ejecuta.

En ReactLand se tiende a evitar los efectos secundarios en las funciones o componentes siempre que sea posible, ya que no encajan bien con la filosofía de React:

- React renderiza la UI cuando cambia el estado (predecible). Los efectos no son predecibles.
- Efectos innecesarios pueden impactar negativamente en el rendimiento.
- Los efectos secundarios descontrolados son difíciles de depurar y testear.

Sin embargo, a pesar de estas dificultades, los efectos secundarios son a menudo inevitables para construir una aplicación. Para ello, existe el hook **useEffect**, una herramienta para manejarlos.

3.4 El hook **useEffect**

En React, existe un hook llamado **useEffect** que sirve para gestionar efectos secundarios de forma controlada, predecible y eficiente. Primero, analicemos la estructura de un **useEffect()** para conocer las partes clave de este hook:

```
useEffect(() => {  
  /* Montaje */  
  
  return () => { /* Desmontaje */ }  
}, [/* dependencias */]);
```

- El **useEffect()** tiene dos parámetros: la función (de montaje) y las dependencias.
- La función del **useEffect** se ejecuta cuando se monta el componente

- El array deps indica las dependencias del `useEffect`
- La función devuelta por el `return` se ejecuta cuando se desmonta el componente

La parte más importante aquí probablemente sea el array de dependencias, así que vamos a explicarlo primero. Las situaciones son las siguientes:

Dependencias	Descripción	Montaje	En cada renderizado
<code>[]</code>	Sólo se ejecuta la primera vez (<code>montaje</code>).	✓	✗
<code>[dep1]</code>	Se ejecuta primera vez y al cambiar.	✓	🔲 Cuando cambia <code>dep1</code>
<code>[dep1, dep2]</code>	Se ejecuta primera vez y al cambiar.	✓	🔲 Cuando cambian <code>dep1</code> y/o <code>dep2</code>
<code>Sin array de dependencias</code>	Se ejecuta primera vez y en cada renderizado.	✓	✓ Siempre

Con esto claro, veamos ahora un código real utilizando un **`useEffect`**. Vamos a crear un temporizador que se ejecutará cada segundo para mostrar la hora actual.

Al inicio del componente `Clock` creamos un estado `time` con la hora actual. Luego, creamos un **`useEffect`** con un array de dependencias vacío, por lo que se ejecutará solo la primera vez que se monta el componente:

```
import { useState, useEffect } from "react";

export function Clock() {
  const [time, setTime] = useState(new Date().toLocaleTimeString());

  useEffect(() => {
    const interval = setInterval(() => {
      setTime(new Date().toLocaleTimeString());
    }, 1000);

    return () => clearInterval(interval);
  }, []);

  return <h1>Hora actual: {time}</h1>;
}
```

En dicho **useEffect** creamos el `setInterval` cada 1 segundo (1000ms) y mutamos el estado `time` para cambiar la hora actual.

Por último, en el `return` devolvemos una función que se ejecutará cuando se desmonte el componente, donde es necesario realizar tareas de limpieza para evitar fugas de memoria o problemas similares.

De esta forma puedes gestionar los efectos secundarios en tus componentes de React de forma controlada. Aunque es un sistema muy potente, ten mucho cuidado y no abuses de `useEffect` ya que puede complicar la lógica, impactar en el rendimiento y muchas veces se puede buscar una alternativa sin efectos secundarios.